Django Database & ORM — Full Notes A to Z

Django 6.0 | English Explanation and Python Examples

Part 1: Introduction to Database and Django ORM

1. What Is a Database?

A database is a system for storing, retrieving, updating, and deleting structured data.

For example, a Student Management System may store:

- Student names and email addresses
- Course names
- Student course enrollments
- Examination results

2. What Is ORM?

ORM stands for **Object-Relational Mapping**.

Django ORM allows developers to work with relational databases using Python code instead of writing SQL for every operation.

SQL:

```
SELECT * FROM students WHERE age >= 18;
```

Django ORM:

```
Student.objects.filter(age__gte=18)
```

3. Django Database Configuration

The `DATABASES` setting in `settings.py` configures database connections.

SQLite example:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Common database backends include SQLite, PostgreSQL, MySQL, and Oracle.

The availability of particular features depends on the Django version, database driver, and backend.

4. What Are Migrations?

Migrations are Django's mechanism for applying model changes to the database schema.

```
python manage.py makemigrations
python manage.py migrate
```

- `makemigrations`: Creates migration files based on model changes.
- `migrate`: Applies migrations to the database.

Additional commands:

```
python manage.py showmigrations
python manage.py sqlmigrate app_name 0001
```

`showmigrations` displays migration status. `sqlmigrate` displays the SQL generated for a particular migration.

Remember: Changing a model does not automatically change the database schema. You generally need to create and apply migrations.

Part 2: Models

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/models/

5. What Is a Model?

A Django Model is a Python class that generally corresponds to a database table.

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField()

    def __str__(self):
        return self.name
```

Here:

- `Student`: Model class
- `name`, `email`, `age`: Model fields
- A `Student` instance: One student record
- `__str__()`: Returns a readable representation of the object

6. Important Model Fields

Field	Purpose
`CharField`	Short or fixed-maximum-length text
`TextField`	Long text
`IntegerField`	Integer values
`PositiveIntegerField`	Positive integer values
`BooleanField`	True/False values
`EmailField`	Email addresses
`URLField`	URLs
`DateField`	Dates

`DateTimeField`	Dates and times
`TimeField`	Times
`DecimalField`	Precise decimal values
`FloatField`	Floating-point values
`UUIDField`	UUID values
`JSONField`	JSON-compatible data
`FileField`	File path/reference
`ImageField`	Image path/reference
`ForeignKey`	Many-to-one relationship
`OneToOneField`	One-to-one

	relationship
`ManyToManyField`	Many-to-many relationship

Example:

```
class Product(models.Model):
    name = models.CharField(max_length=150)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    is_available = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

For financial values, `DecimalField` is generally preferable to `FloatField`.

7. Common Field Options

`null=True`

Allows the database to store `NULL`.

```
age = models.IntegerField(null=True, blank=True)
```

`blank=True`

Allows a field to be empty during Django validation.

`default`

Defines a default value.

```
is_active = models.BooleanField(default=True)
```

`unique=True`

Prevents duplicate values in the field.

```
email = models.EmailField(unique=True)
```

`db_index=True`

Requests a database index for a field.

```
email = models.EmailField(db_index=True)
```

`choices`

Restricts a field to a predefined set of choices.

```
class Student(models.Model):
    class Status(models.TextChoices):
        ACTIVE = "active", "Active"
        INACTIVE = "inactive", "Inactive"

    name = models.CharField(max_length=100)
    status = models.CharField(
        max_length=10,
        choices=Status,
        default=Status.ACTIVE,
    )
```

Get the display label:

```
student.get_status_display()
```

`primary_key=True`

Defines the primary key used to identify a record.

```
import uuid

student_id = models.UUIDField(
    primary_key=True,
    default=uuid.uuid4,
    editable=False,
)
```

Django can create a primary key automatically. If you define a custom primary key, Django generally does not add another `id` field.

8. Difference Between `null` and `blank`

Option	Purpose
`null=True`	Allows NULL values in the database
`blank=True`	Allows empty values during validation

For example:

```
description = models.TextField(blank=True)
```

For text fields, an empty string is generally preferred over `NULL` unless nullable text has a specific purpose.

9. Model Meta Options

The inner `Meta` class defines model metadata.

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        db_table = "store_product"
        ordering = ["name"]
        verbose_name = "Product"
        verbose_name_plural = "Products"
```

Important options:

- `db_table` : Database table name
- `ordering` : Default ordering
- `verbose_name` : Singular display name

- `verbose_name_plural`: Plural display name
- `constraints`: Database-level constraints
- `indexes`: Index configuration
- `unique_together`: Older composite uniqueness option; `UniqueConstraint` is generally preferred in new code

10. Constraints

Constraints help enforce data integrity at the database level.

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            )
        ]
```

This creates a rule preventing negative prices when the database backend supports the constraint.

To enforce uniqueness across multiple fields:

```
class Enrollment(models.Model):
    student = models.ForeignKey("Student", on_delete=models.CASCADE)
    course = models.ForeignKey("Course", on_delete=models.CASCADE)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["student", "course"],
                name="unique_student_course",
            )
        ]
```

Part 3: Model Relationships

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/models/

11. One-to-Many / Many-to-One

One department can have many students, while each student belongs to one department in this example.

```
class Department(models.Model):
    name = models.CharField(max_length=100)


class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.ForeignKey(
        Department,
        on_delete=models.CASCADE,
        related_name="students",
    )
```

Usage:

```
department.students.all()
student.department
student.department_id
```

A `ForeignKey` normally requires an `on_delete` option.

Important `on_delete` options:

- `CASCADE`: Deletes related objects when the referenced object is deleted.
- `PROTECT`: Prevents deletion when protected related objects exist.
- `RESTRICT`: Restricts deletion when related objects exist, subject to Django's deletion rules.
- `SET_NULL`: Sets the foreign key to `NULL`; requires `null=True`.
- `SET_DEFAULT`: Sets the foreign key to its default value.
- `SET(...)`: Sets a specified value or callable result.
- `DO_NOTHING`: Django takes no deletion action; a database constraint may raise an error.

12. One-to-One Relationship

A user can have one profile.

```
from django.conf import settings
from django.db import models
```

```
class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
```

Usage:

```
profile.user
user.profile
```

Using `settings.AUTH_USER_MODEL` is recommended when referring to the project's user model, because the project may use a custom user model.

13. Many-to-Many Relationship

A student can enroll in multiple courses, and each course can have multiple students.

```
class Course(models.Model):
    title = models.CharField(max_length=100)


class Student(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(
        Course,
        related_name="students",
    )
```

Usage:

```
student.courses.add(course1, course2)
student.courses.remove(course1)
student.courses.clear()
student.courses.set([course1, course2])

student.courses.all()
course.students.all()
```

Django normally creates an intermediate table for a many-to-many relationship.

14. Through Model

Use a `through` model when a relationship needs additional data.

```
class Student(models.Model):
    name = models.CharField(max_length=100)


class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(
        Student,
        through="Enrollment",
        related_name="courses",
    )


class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(auto_now_add=True)
    grade = models.CharField(max_length=5, blank=True)
```

The `Enrollment` model stores additional information such as the enrollment date and grade.

15. Reverse Relationships

Without `related_name`, Django generally uses `<modelname>_set` for reverse relationships.

```
class Book(models.Model):
    author = models.ForeignKey("Author", on_delete=models.CASCADE)
```

Usage:

```
author.book_set.all()
```

If `related_name="books"` is defined:

```
author.books.all()
```

16. Recursive Relationships

A model can have a relationship with itself.

```
class Employee(models.Model):
    name = models.CharField(max_length=100)
    manager = models.ForeignKey(
        "self",
        null=True,
        blank=True,
        on_delete=models.SET_NULL,
        related_name="team_members",
    )
```

Part 4: Making Queries and QuerySets

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/queries/

17. What Is a QuerySet?

A QuerySet is a collection-like object representing the results of a database query.

```
Student.objects.all()
```

QuerySets are generally **lazy**: creating one does not immediately execute the query. Evaluation may occur when iterating, converting it to a list, calling `len()`, evaluating its truth value, or using certain terminal operations.

18. Retrieve All Records

```
Student.objects.all()
```

19. Filter Records

Returns records matching specified conditions.

```
Student.objects.filter(age=20)
Student.objects.filter(age__gte=18)
Student.objects.filter(name__startswith="A")
```

20. Exclude Records

Excludes records matching specified conditions.

```
Student.objects.exclude(age__lt=18)
```

21. Get a Single Object

```
student = Student.objects.get(id=1)
```

Possible exceptions:

- `Student.DoesNotExist`
- `Student.MultipleObjectsReturned`

Example with exception handling:

```
try:
    student = Student.objects.get(id=1)
except Student.DoesNotExist:
    student = None
```

Use `get()` when the query is expected to return exactly one object.

22. QuerySet Lookups

Lookup	Meaning
`exact`	Exact match
`iexact`	Case-insensitive exact match
`contains`	Contains a substring
`icontains`	Case-insensitive substring match

`startswith`	Starts with a value
`istartswith`	Case-insensitive prefix match
`endswith`	Ends with a value
`gt`	Greater than
`gte`	Greater than or equal to
`lt`	Less than
`lte`	Less than or equal to
`in`	Matches a collection
`range`	Falls within a range
`isnull`	Checks whether a value is NULL

`year`	Filters by year
`date`	Filters by date

Examples:

```
Student.objects.filter(age__gt=18)
Student.objects.filter(age__range=(18, 25))
Student.objects.filter(name__icontains="faruk")
Student.objects.filter(id__in=[1, 2, 3])
Student.objects.filter(birth_date__year=2000)
Student.objects.filter(email__isnull=True)
```

23. AND Conditions

Multiple conditions in one `filter()` are combined using AND.

```
Student.objects.filter(age__gte=18, is_active=True)
```

24. OR Conditions with Q Objects

```
from django.db.models import Q

Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)
```

Operators:

- `&`: AND
- `|`: OR
- `~`: NOT

Example:

```
Student.objects.filter(
    Q(name__icontains="omar") |
    Q(email__icontains="omar")
)
```

25. Ordering

```
Student.objects.order_by("name")
Student.objects.order_by("-age")
```

The `-` prefix indicates descending order.

Ordering by multiple fields:

```
Student.objects.order_by("department__name", "-age")
```

26. Limit and Offset

```
Student.objects.all()[:10]
Student.objects.all()[10:20]
```

These operations are useful for limiting results and implementing pagination.

Negative indexing and ordinary reverse Python slicing are not supported by QuerySet slicing. After slicing, further filtering or ordering may also be restricted.

27. Values and Values List

Retrieve only selected fields:

```
Student.objects.values("id", "name")
```

This returns dictionary-like records.

```
Student.objects.values_list("name", flat=True)
```

This returns a flat iterable of names.

28. Distinct Results

Use `distinct()` to remove duplicate results.

```
Student.objects.values("department_id").distinct()
```

The fields used for distinct selection and the effects of ordering can depend on the database backend.

29. Exists, Count, First, and Last

```
Student.objects.count()
Student.objects.exists()
```

```
Student.objects.first()
Student.objects.last()
```

- `count()` : Returns the number of records.
- `exists()` : Checks whether a matching record exists.
- `first()` : Returns the first object or `None` .
- `last()` : Returns the last object or `None` .

Use `exists()` when you only need to know whether a result exists. Consider whether a separate query is necessary if you will later use the full QuerySet anyway.

30. Get or Create

```
student, created = Student.objects.get_or_create(
    email="student@example.com",
    defaults={
        "name": "Rahim",
        "age": 20,
    },
)
```

- `student` : The retrieved or newly created object
- `created` : Whether a new object was created

Database uniqueness constraints are important for avoiding race conditions.

31. Update or Create

```
student, created = Student.objects.update_or_create(
    email="student@example.com",
    defaults={
        "name": "Rahim",
        "age": 21,
    },
)
```

32. Create Records

```
student = Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
    age=20,
)
```

Alternatively:

```
student = Student(
    name="Rahim",
    email="rahim@example.com",
    age=20,
)
student.save()
```

33. Update Records

Update multiple records at once:

```
Student.objects.filter(age=20).update(age=21)
```

`QuerySet.update()` performs a direct SQL update. It does not call each instance's `save()` method or trigger the usual model save signals.

Update one object:

```
student = Student.objects.get(pk=1)
student.age = 22
student.save()
```

34. Delete Records

```
Student.objects.filter(is_active=False).delete()
```

Delete one object:

```
student.delete()
```

Always verify the filter before deleting data. Calling `all().delete()` can delete every record in the table.

35. F Expressions

`F()` expressions refer to database field values and allow database-side calculations.

```
from django.db.models import F

Product.objects.filter(pk=1).update(
    stock=F("stock") - 1
)
```

This avoids reading a value into Python and then writing it back as a separate operation.

36. Conditional Queries with Case and When

```
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    level=Case(
        When(age__lt=18, then=Value("Minor")),
        default=Value("Adult"),
        output_field=CharField(),
    )
)
```

37. Bulk Operations

```
Student.objects.bulk_create([
    Student(name="A", email="a@example.com", age=20),
    Student(name="B", email="b@example.com", age=21),
])
```

```
Student.objects.bulk_update(
    students,
    ["age"],
)
```

Bulk operations can be efficient for large numbers of records. However, they do not follow the normal instance-level `save()` and signal behavior. Generated primary keys and other details can also vary by backend and operation.

Part 5: Aggregation and Annotation

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/aggregation/

38. What Is Aggregation?

Aggregation calculates summary values across multiple records.

```
from django.db.models import Avg, Count, Max, Min, Sum
```

Example:

```
Student.objects.aggregate(Avg("age"))
```

39. Common Aggregate Functions

Function	Purpose
`Count()`	Counts records
`Sum()`	Calculates a total
`Avg()`	Calculates an average
`Max()`	Finds the maximum
`Min()`	Finds the minimum

Example:

```
Product.objects.aggregate(
    total_products=Count("id"),
    total_stock=Sum("stock"),
    average_stock=Avg("stock"),
    highest_stock=Max("stock"),
    lowest_stock=Min("stock"),
)
```

`aggregate()` generally returns a dictionary.

40. What Is Annotation?

`annotate()` adds calculated values to individual objects or groups.

```
from django.db.models import Count

departments = Department.objects.annotate(
    student_count=Count("students")
```

```
)

for department in departments:
    print(department.name, department.student_count)
```

41. Aggregation vs Annotation

aggregate()	**annotate()**
Produces an overall summary	Adds summary values to objects or groups
Generally returns a dictionary	Returns an annotated QuerySet
Example: Total number of students	Example: Number of students per department

42. Grouping Results

```
Student.objects.values("department_id").annotate(
    total_students=Count("id")
)
```

This calculates the number of students in each department.

43. Order of Filter and Annotation

```
Publisher.objects.annotate(
    book_count=Count("book")
).filter(book_count__gt=5)
```

Changing the order of `filter()` and `annotate()` can change the results. For complex aggregation, inspect the generated SQL and joins.

44. Conditional Aggregation

```
from django.db.models import Count, Q

Department.objects.annotate(
    active_students=Count(
        "students",
        filter=Q(students__is_active=True),
    )
)
```

45. Distinct Count

```
Department.objects.annotate(
    student_count=Count("students", distinct=True)
)
```

`distinct=True` can be useful when joins create duplicate rows. Understand the query structure before using it.

Part 6: Database Search

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/search/

46. Basic Text Search

```
Student.objects.filter(name__icontains="rahim")
```

The case sensitivity of `contains` and `icontains` can depend on the database backend and collation.

47. Search Across Multiple Fields

```
from django.db.models import Q
```

```
Student.objects.filter(
    Q(name__icontains="rahim") |
    Q(email__icontains="rahim")
)
```

48. PostgreSQL Full-Text Search

When using PostgreSQL, Django provides `SearchVector`, `SearchQuery`, and `SearchRank`.

```
from django.contrib.postgres.search import (
    SearchQuery,
    SearchVector,
    SearchRank,
)

articles = Article.objects.annotate(
    search=SearchVector("title", "body")
).filter(
    search=SearchQuery("django database")
)
```

Rank results by relevance:

```
query = SearchQuery("django database")

articles = (
    Article.objects
    .annotate(
        rank=SearchRank(
            SearchVector("title", "body"),
            query,
        )
    )
    .filter(rank__gt=0)
    .order_by("-rank")
)
```

These PostgreSQL-specific search APIs require the PostgreSQL backend.

Part 7: Managers

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/managers/

49. What Is a Manager?

A Manager provides an interface for accessing a model's database queries.

```
Student.objects.all()
```

Here, `objects` is the conventional default manager name.

50. Custom Manager

```
class ActiveStudentManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)


class Student(models.Model):
    name = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True)

    objects = models.Manager()
    active_students = ActiveStudentManager()
```

Usage:

```
Student.objects.all()
Student.active_students.all()
```

Custom managers help organize reusable query logic.

51. Custom Manager Methods

```
class StudentManager(models.Manager):
    def adults(self):
        return self.get_queryset().filter(age__gte=18)


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()

    objects = StudentManager()
```

Usage:

```
Student.objects.adults()
```

52. `QuerySet.as_manager()`

Expose reusable QuerySet methods through a Manager.

```
class StudentQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)

    def adults(self):
        return self.filter(age__gte=18)


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.PositiveIntegerField()
    is_active = models.BooleanField(default=True)

    objects = StudentQuerySet.as_manager()
```

```
Student.objects.active().adults()
```

53. `_default_manager` and `_base_manager`

- `_default_manager`: The model's configured default manager.
- `_base_manager`: A manager Django uses for certain internal operations and related-object access.

When implementing soft deletion or filtered managers, understand the distinction. A base manager that filters out records can interfere with related-object access.

Part 8: Performing Raw SQL Queries

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/sql/

54. Why Use Raw SQL?

Django ORM handles most common database operations. Raw SQL can be useful for complex or database-specific operations.

55. `Manager.raw()`

```
students = Student.objects.raw(
    "SELECT * FROM school_student WHERE age >= %s",
    [18],
)

for student in students:
    print(student.name)
```

Important points:

- `%s` is used as a parameter placeholder.
- Parameters are supplied separately in a list.
- User input is not directly concatenated into the SQL string.

`raw()` generally returns an iterable of model instances. The SQL result must include the model's primary key.

56. `connection.cursor()`

For more direct SQL execution:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT COUNT(*) FROM school_student WHERE age >= %s",
        [18],
    )
    result = cursor.fetchone()
```

`result` is generally a tuple.

For multiple databases, use a connection from `connections["alias"]`.

57. Preventing SQL Injection

Unsafe approach:

```
query = f"SELECT * FROM school_student WHERE name = '{name}'"
```

This can expose the application to SQL injection.

Safer approach:

```
cursor.execute(
    "SELECT * FROM school_student WHERE name = %s",
```

```
        [name],
    )
```

Use parameterized queries. Table and column names cannot generally be bound as ordinary value parameters. If they must be dynamic, use a trusted allowlist.

Part 9: Database Transactions

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/transactions/

58. What Is a Transaction?

A transaction is a logical unit of one or more database operations.

For example, a bank transfer must debit one account and credit another account consistently.

59. `transaction.atomic()`

```
from django.db import transaction

with transaction.atomic():
    account1.balance -= 100
    account1.save()

    account2.balance += 100
    account2.save()
```

When the transaction block completes successfully, changes are committed. Database errors can cause the transaction to roll back.

This example demonstrates the concept. A real financial transfer also requires balance validation, concurrency protection, and database-level integrity.

60. Atomic Function Decorator

```
from django.db import transaction

@transaction.atomic
def create_order():
    # Related database operations
    ...
```

61. `ATOMIC_REQUESTS`

Example database configuration:

```
DATABASES = {
    "default": {
        # Backend settings...
        "ATOMIC_REQUESTS": True,
    }
}
```

This can wrap the execution of each view request in a transaction for the relevant database.

It does not automatically include every stage of request processing. Middleware and template response processing, for example, can be outside the transaction boundary.

62. `transaction.on_commit()`

Run a callback after a successful transaction commit:

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(...)

    transaction.on_commit(
        lambda: send_order_notification(order.pk)
    )
```

The callback does not run if the transaction rolls back.

This is useful for sending emails, updating caches, or triggering external side effects after a successful commit.

63. Nested Atomic Blocks

```
with transaction.atomic():
    order = Order.objects.create(...)

    with transaction.atomic():
        order_item = OrderItem.objects.create(...)
```

Nested atomic blocks generally use savepoints when supported. If the outer transaction rolls back, changes from a successful inner block can also be rolled back.

64. Exception Handling

Continuing to run queries inside a transaction after a database error can cause problems.

A safer pattern:

```
from django.db import IntegrityError, transaction

try:
    with transaction.atomic():
        create_unique_record()
except IntegrityError:
    handle_duplicate_record()
```

The exception is caught outside the `atomic()` block, allowing Django to handle the failed transaction state correctly.

65. Important Limitations

- A transaction generally operates on one database connection.
- Changes across multiple database aliases are not automatically committed as one atomic transaction.
- External API calls and emails cannot be rolled back by a database transaction.
- For concurrency-sensitive operations, consider `select_for_update()`, constraints, or atomic updates.

Part 10: Multiple Databases

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/multi-db/

66. What Are Multiple Databases?

A Django project can use multiple database connections.

For example:

- `default`: Main application database
- `analytics`: Analytics data
- `legacy`: An older database

67. Database Configuration

```
DATABASES = {
    "default": {
```

```
            "ENGINE": "django.db.backends.sqlite3",
            "NAME": BASE_DIR / "db.sqlite3",
        },
        "analytics": {
            "ENGINE": "django.db.backends.sqlite3",
            "NAME": BASE_DIR / "analytics.sqlite3",
        },
    }
```

In production, each database requires appropriate engine, credentials, and connection settings.

68. `using()`

```
Student.objects.using("analytics").all()
```

69. Select a Database When Saving

```
student = Student(name="Rahim", age=20)
student.save(using="analytics")
```

70. `db_manager()`

Choose a database for manager methods:

```
Student.objects.db_manager("analytics").create(
    name="Rahim",
    age=20,
)
```

71. Database Router

A router determines which database should handle reads, writes, relations, and migrations.

```
class AnalyticsRouter:
    route_app_labels = {"analytics_app"}

    def db_for_read(self, model, **hints):
        if model._meta.app_label in self.route_app_labels:
            return "analytics"
        return None
```

```
        def db_for_write(self, model, **hints):
            if model._meta.app_label in self.route_app_labels:
                return "analytics"
            return None

        def allow_relation(self, obj1, obj2, **hints):
            if (
                obj1._state.db == "analytics"
                or obj2._state.db == "analytics"
            ):
                return obj1._state.db == obj2._state.db
            return None

        def allow_migrate(self, db, app_label, **hints):
            if app_label in self.route_app_labels:
                return db == "analytics"
            return None
```

In `settings.py`:

```
DATABASE_ROUTERS = ["project.routers.AnalyticsRouter"]
```

This is an illustrative example. A real router needs explicit policies for all relevant apps, relations, and migrations.

72. Cross-Database Relationships

Django does not automatically support ordinary relational foreign keys or many-to-many relationships across separate databases.

Plan relationships, migrations, and transaction behavior carefully before adopting multiple databases.

Part 11: Tablespaces

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/tablespaces/

73. What Is a Tablespace?

A tablespace is a database storage organization concept. Some database backends use tablespaces to control where tables and indexes are stored.

Example model tablespace:

```
class Student(models.Model):
    name = models.CharField(max_length=100)

    class Meta:
        db_tablespace = "student_tablespace"
```

Example index tablespace:

```
class Student(models.Model):
    email = models.EmailField(db_index=True)

    class Meta:
        db_tablespace = "student_tablespace"
        indexes = [
            models.Index(
                fields=["email"],
                db_tablespace="student_index_tablespace",
            )
        ]
```

Tablespace support is backend-specific. For example, SQLite may not support this configuration in the expected way.

Part 12: Database Access Optimization

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/optimization/

74. Why Is Optimization Important?

Slow database queries increase API response times and server resource usage.

Before optimizing, measure query counts, execution plans, table sizes, and real workloads.

75. QuerySet Laziness

```
students = Student.objects.filter(age__gte=18)
```

This line generally does not immediately execute the query.

```
for student in students:
    print(student.name)
```

The QuerySet may be evaluated during iteration.

76. QuerySet Caching

An evaluated QuerySet can reuse its result cache when the same QuerySet instance is used again.

```
students = Student.objects.all()

list(students)
list(students)
```

The second evaluation can reuse the cached result. Creating a separate QuerySet may execute another query.

77. N+1 Query Problem

Consider:

```
students = Student.objects.all()

for student in students:
    print(student.department.name)
```

If each department is fetched separately, the code may generate many queries.

Solution: `select_related()`

```
students = Student.objects.select_related("department")
```

`select_related()` generally uses SQL joins for `ForeignKey` and `OneToOneField` relationships.

78. `prefetch_related()`

For collections such as many-to-many and reverse relationships:

```
departments = Department.objects.prefetch_related("students")

for department in departments:
    for student in department.students.all():
        print(student.name)
```

`prefetch_related()` generally uses additional queries and joins the results in Python.

Difference

`select_related()`	`prefetch_related()`
Generally uses SQL JOIN	Generally uses separate queries and Python-side matching
Best suited to ForeignKey and OneToOne	Supports ManyToMany, reverse relationships, and more
Can fetch related columns in one query	May execute multiple queries

79. `only()` and `defer()`

Retrieve only selected fields:

```
Student.objects.only("id", "name")
```

Defer a large field:

```
Article.objects.defer("large_content")
```

Accessing a deferred field later can trigger an additional query. Use these methods only when justified by actual performance measurements.

80. `values()` and `values_list()`

Retrieve selected data instead of full model instances:

```
Student.objects.values("id", "name")
```

```
Student.objects.values_list("name", flat=True)
```

These methods can be useful for reporting and data serialization.

81. `iterator()`

Process large QuerySets without retaining the normal QuerySet result cache:

```
for student in Student.objects.all().iterator(chunk_size=1000):
    process_student(student)
```

This can help with large data processing. Behavior may vary with the backend and prefetch configuration.

82. `explain()`

Inspect how the database plans to execute a query:

```
print(
    Student.objects.filter(age__gte=18).explain()
)
```

The execution plan can reveal table scans, index usage, and joins.

83. Indexing

```
class Student(models.Model):
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField(db_index=True)
```

Indexes can improve searches and filtering, but:

- Too many indexes can slow write operations.
- Indexes consume storage.
- Index selection should reflect real query patterns.

Composite index:

```
class Meta:
    indexes = [
        models.Index(fields=["department", "age"]),
    ]
```

The order of fields in a composite index matters.

84. Atomic Updates with `F()`

```
from django.db.models import F

Product.objects.filter(pk=1).update(
    stock=F("stock") - 1
)
```

This reduces the risk of application-side read-modify-write races. Database constraints or conditional updates may still be needed to prevent negative stock.

85. Bulk Create and Bulk Update

```
Student.objects.bulk_create(student_objects)
```

```
Student.objects.bulk_update(
    student_objects,
    ["age"],
)
```

Bulk operations can be more efficient than calling `save()` for every record.

86. `count()`, `exists()`, and `contains()`

```
Student.objects.count()
Student.objects.filter(age__gte=18).exists()
Student.objects.contains(student)
```

- Use `count()` when you need the count.
- Use `exists()` when you only need to know whether any matching record exists.
- Use `contains()` when checking whether a specific object belongs to a QuerySet.

Consider whether an additional query is necessary when the results will be reused.

87. Pagination

```
from django.core.paginator import Paginator

students = Student.objects.order_by("id")
paginator = Paginator(students, 20)

page = paginator.get_page(1)
```

Use stable ordering before paginating.

Part 13: Database Instrumentation

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/instrumentation/

88. What Is Database Instrumentation?

Database instrumentation means monitoring database queries and connection activity.

It helps analyze query execution, performance issues, and database behavior.

89. Query Logging

During development, Django's SQL logging can show executed database queries.

```
LOGGING = {
    "version": 1,
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
        },
    },
    "loggers": {
        "django.db.backends": {
            "handlers": ["console"],
            "level": "DEBUG",
        },
    },
}
```

This is a development-oriented example. In production, consider log volume, sensitive information, and performance impact.

90. `connection.execute_wrapper()`

Monitor query execution using a wrapper:

```
from django.db import connection


def query_wrapper(execute, sql, params, many, context):
    print("SQL:", sql)

    return execute(sql, params, many, context)


with connection.execute_wrapper(query_wrapper):
    list(Student.objects.all())
```

Real instrumentation can record duration, errors, and query metadata. Avoid logging passwords, tokens, or personal data.

Part 14: Fixtures

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/fixtures/

91. What Is a Fixture?

A fixture is prepared data that can be loaded into a database.

Fixtures are useful for development, testing, and initial sample data.

92. Fixture Formats

Django fixtures commonly use JSON, XML, or YAML formats, subject to the available serializers and dependencies.

JSON example:

```
[
  {
    "model": "school.student",
    "pk": 1,
    "fields": {
      "name": "Rahim",
      "email": "rahim@example.com",
      "age": 20
    }
```

```
  }
]
```

Here, `school` is the app label and `student` is the model name.

93. Load Fixtures

```
python manage.py loaddata students.json
```

94. Dump Fixtures

```
python manage.py dumpdata school.Student --indent 2
```

Save output to a file:

```
python manage.py dumpdata school.Student --indent 2 > students.json
```

95. Fixture Precautions

- Check privacy before exporting production data.
- Primary key conflicts can occur.
- Foreign key and many-to-many dependencies must be handled correctly.
- Consider transaction and signal behavior when loading fixtures.
- Never include sensitive credentials in fixtures.

Part 15: Relationship API Examples

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/examples/

96. Many-to-One Example

```
class Reporter(models.Model):
    name = models.CharField(max_length=100)


class Article(models.Model):
    headline = models.CharField(max_length=200)
    reporter = models.ForeignKey(
        Reporter,
        on_delete=models.CASCADE,
        related_name="articles",
    )
```

Create records:

```
reporter = Reporter.objects.create(name="Rahim")

Article.objects.create(
    headline="Django ORM",
    reporter=reporter,
)
```

Forward access:

```
article = Article.objects.first()
article.reporter
```

Reverse access:

```
reporter.articles.all()
```

97. One-to-One Example

```
class Place(models.Model):
    name = models.CharField(max_length=100)


class Restaurant(models.Model):
    place = models.OneToOneField(
        Place,
        on_delete=models.CASCADE,
        related_name="restaurant",
    )
    serves_pizza = models.BooleanField(default=False)
```

```
place = Place.objects.create(name="Central Place")

restaurant = Restaurant.objects.create(
    place=place,
    serves_pizza=True,
)

place.restaurant
restaurant.place
```

Accessing a reverse one-to-one relationship can raise a related-object exception if the associated object does not exist.

98. Many-to-Many Example

```
class Publication(models.Model):
    title = models.CharField(max_length=100)


class Article(models.Model):
    headline = models.CharField(max_length=200)
    publications = models.ManyToManyField(Publication)
```

```
publication = Publication.objects.create(title="Tech Magazine")

article = Article.objects.create(headline="Learning Django")
article.publications.add(publication)

article.publications.all()
publication.article_set.all()
```

The relevant model instances must be saved before creating a many-to-many relationship.

Part 16: Advanced ORM Concepts

99. `Subquery()` and `OuterRef()`

Use one query inside another query:

```
from django.db.models import OuterRef, Subquery

latest_article = Article.objects.filter(
    reporter=OuterRef("pk")
).order_by("-id")

reporters = Reporter.objects.annotate(
    latest_headline=Subquery(
        latest_article.values("headline")[:1]
    )
)
```

This annotates each Reporter with the headline of their latest Article, based on the example's ordering.

100. `Exists()`

Check whether related records exist:

```
from django.db.models import Exists, OuterRef

articles = Article.objects.filter(
    reporter=OuterRef("pk")
)

reporters = Reporter.objects.annotate(
    has_articles=Exists(articles)
)
```

101. Database Functions

Django database functions perform calculations in the database.

```
from django.db.models.functions import Lower

students = Student.objects.annotate(
    lowercase_name=Lower("name")
)
```

Function availability and behavior can vary by database backend.

102. `select_for_update()`

Lock selected rows within a transaction when the database backend supports it:

```
from django.db import transaction

with transaction.atomic():
    account = (
        Account.objects
        .select_for_update()
        .get(pk=1)
    )

    account.balance -= 100
    account.save()
```

This is useful for concurrency-sensitive operations. Not all database backends, including SQLite, support row-level locking in the same way.

103. Signals and `save()`

Signals can respond to model saves and deletions. However, putting all business logic in signals is often undesirable because it can make the code flow difficult to understand.

Important differences:

- `save()` generally saves one model instance.
- `QuerySet.update()` performs a direct database update.
- `bulk_create()` and `bulk_update()` do not follow normal instance-level save behavior.

Understand which validations, overrides, and signals run for each operation.

Part 17: Common Mistakes

104. Forgetting Migrations

Problem: A field is added to a model, but the corresponding database column does not exist.

Solution:

```
python manage.py makemigrations
python manage.py migrate
```

105. Confusing `get()` and `filter()`

```
Student.objects.get(pk=1)
```

Expects one object and raises an exception if zero or multiple objects match.

```
Student.objects.filter(pk=1)
```

Returns a QuerySet.

106. Confusing `null` and `blank`

`null` relates to database values; `blank` relates to validation.

107. N+1 Queries

When accessing related objects inside a loop, check whether `select_related()` or `prefetch_related()` is appropriate.

108. Unsafe Raw SQL

Never concatenate untrusted user input directly into SQL strings. Use parameterized queries.

109. Unnecessary `.all()`

In many cases, you can use a QuerySet directly:

```
Student.objects.filter(...)
```

`.all()` is not inherently wrong, but it can be unnecessary in some expressions.

110. Incorrect Expectations About Bulk Operations

Do not assume `update()`, `bulk_create()`, or `bulk_update()` automatically execute instance-level `save()` overrides, validation, or signals.

111. Indexing Every Field

Indexes can improve performance, but unnecessary indexes increase storage and write costs. Use query plans and measurements to guide decisions.

112. External Side Effects Inside Transactions

Database rollback cannot undo an email or external API call. Consider `transaction.on_commit()` when the action should happen only after a successful commit.

Part 18: Best Practices

1. Use meaningful model and field names.
2. Enforce data integrity with appropriate `UniqueConstraint` and `CheckConstraint` rules.
3. Understand `on_delete` behavior before selecting a deletion strategy.
4. Keep QuerySet chains readable.
5. Check for N+1 queries when accessing related objects.
6. Consider bulk operations and `iterator()` when processing large datasets.
7. Measure performance and use `explain()` before optimizing queries.
8. Use parameterized queries when writing Raw SQL.
9. Plan routers, migrations, and relationships carefully when using multiple databases.

10. Consider database constraints and locking strategies for concurrency-sensitive operations.
11. Avoid exposing sensitive data in production logs.
12. Never put production credentials in fixtures.
13. Maintain a database backup and recovery strategy.
14. Learn to inspect generated SQL.
15. Consult the official documentation for version-specific and backend-specific behavior.

Part 19: Interview Questions and Quick Revision

Beginner

1. What is Django ORM?

An abstraction that allows developers to work with relational databases using Python objects and QuerySets.

2. What is a Model?

A Python class representing database data and related behavior.

3. What is a Migration?

A versioned mechanism for applying model changes to the database schema.

4. What is the difference between `get()` and `filter()`?

`get()` expects one object; `filter()` returns a QuerySet.

5. What is the difference between `null=True` and `blank=True`?

`null` concerns database values; `blank` concerns validation.

Intermediate

6. What are `select_related()` and `prefetch_related()`?

Techniques for optimizing queries involving related objects.

7. What is the difference between `annotate()` and `aggregate()`?

`annotate()` adds calculated values to objects or groups; `aggregate()` calculates an overall summary.

8. What is an `F()` expression?

It allows calculations using database field values without first loading those values into Python.

9. Why use a Custom Manager?

To create reusable query logic and a model-specific query interface.

10. What is `transaction.atomic()`?

It executes database operations within a transaction block.

Advanced

11. What is the N+1 query problem?

A performance issue in which a main query is followed by additional queries for individual results.

12. What is the difference between `QuerySet.update()` and `save()`?

`update()` performs a direct SQL update; `save()` saves a model instance.

13. How do you prevent SQL injection?

Use parameterized queries and avoid concatenating untrusted input into SQL strings.

14. Why use `explain()`?

To inspect the database's query execution plan.

15. What is `select_for_update()`?

It locks selected rows inside a transaction on supported database backends.

16. What is `transaction.on_commit()`?

It runs a callback after a successful database transaction commit.

17. Can foreign keys span multiple databases?

Django does not automatically support ordinary relational foreign keys across separate databases.

18. What are the limitations of `bulk_create()`?

It does not follow normal instance-level `save()` and signal behavior.

19. Why are QuerySets lazy?

They separate QuerySet construction from query execution, allowing Django to avoid executing queries until the results are needed.

20. When should database indexes be used?

When real query patterns and execution plans indicate that an index will be beneficial. Indexes should not be added to every field without considering their costs.

Final Revision Checklist

- Models and field types
- `null`, `blank`, `default`, and `choices`
- `Meta`, constraints, and indexes
- `ForeignKey`, `OneToOneField`, and `ManyToManyField`
- `related_name` and reverse relationships
- `filter()`, `exclude()`, and `get()`
- Lookup expressions and `Q`
- `order_by()`, slicing, and `values()`
- Create, update, delete, and bulk operations
- `F()`, `Subquery()`, `OuterRef()`, and `Exists()`
- `aggregate()` and `annotate()`
- Search and PostgreSQL full-text search
- Custom Managers and QuerySets
- Raw SQL and SQL injection prevention
- Transactions, savepoints, and `on_commit()`
- Multiple databases and database routers
- Tablespaces
- QuerySet laziness and caching
- N+1 problems and query optimization
- `select_related()` and `prefetch_related()`
- `explain()`, instrumentation, and logging
- Fixtures
- Relationship API
- Database constraints and concurrency
- Interview questions

Official Django 6.0 Documentation

- Models: https://docs.djangoproject.com/en/6.0/topics/db/models/
- Making queries: https://docs.djangoproject.com/en/6.0/topics/db/queries/
- Aggregation: https://docs.djangoproject.com/en/6.0/topics/db/aggregation/
- Search: https://docs.djangoproject.com/en/6.0/topics/db/search/
- Managers: https://docs.djangoproject.com/en/6.0/topics/db/managers/
- Raw SQL: https://docs.djangoproject.com/en/6.0/topics/db/sql/

- Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/
- Multiple databases: https://docs.djangoproject.com/en/6.0/topics/db/multi-db/
- Tablespaces: https://docs.djangoproject.com/en/6.0/topics/db/tablespaces/
- Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Instrumentation: https://docs.djangoproject.com/en/6.0/topics/db/instrumentation/
- Fixtures: https://docs.djangoproject.com/en/6.0/topics/db/fixtures/
- Relationship examples: https://docs.djangoproject.com/en/6.0/topics/db/examples/

Recommended study order: Start with Models and Relationships, then learn QuerySets, Aggregation, and Managers. Next, study Transactions, Raw SQL, Multiple Databases, and Optimization. Practicing each topic in your own Django project is one of the most effective ways to learn it.